

DevOps Interview Prep (L&T): Git, Docker, Kubernetes, Ansible

Oct 4, 2026 · @Bhashkar

Twenty-three questions, grouped Git → Docker → Kubernetes → Ansible; "Explain git branching strategy" was asked twice, so it is answered once, as Q3. Each answer follows the order interviewers expect at 9 years: a one-line definition, how it works, a working example, then trade-offs. The **In the interview** lines are templates: swap in your own project, cloud and numbers before you use them.

Part 1 · Git

Q1. What is git cherry-pick?

`git cherry-pick` copies the changes from one or more existing commits onto your current branch as new commits, without merging the rest of the source branch. The copy has the same diff and message but a new SHA, because its parent is different.

Typical uses:

- **Hotfix backport:** a fix merged to `main` must also go to `release/2.3`, or the reverse.
- **Wrong branch:** a commit landed on `develop` but belongs on a feature branch.
- **Selective release:** ship one finished fix without the unfinished work around it.

Before:

```
main:      A---B---C
              \
feature:    D---E---F
```

After `git checkout main && git cherry-pick F`

```
main:      A---B---C---F'      F' = copy of F with a new SHA
              \
feature:    D---E---F      F itself is unchanged
```

```

git checkout release/2.3
git cherry-pick 3f2a9c1                # one commit
git cherry-pick -x 3f2a9c1            # appends "(cherry picked from commit
...)\" for traceability
git cherry-pick 3f2a9c1 8b7d6e5        # several commits, applied in order
git cherry-pick A^..B                  # a range, A included (A..B excludes A)
git cherry-pick -n 3f2a9c1            # --no-commit: stage only, commit
yourself
git cherry-pick -m 1 <merge-sha>      # a merge commit: diff against its first
parent

# if it conflicts
git status                            # fix the listed files, then
git add <files> && git cherry-pick --continue
git cherry-pick --skip                 # drop this commit, carry on with the
rest
git cherry-pick --abort                # back to where you started

```

Watch-outs:

- It duplicates history. Repeated cherry-picks between long-lived branches make logs confusing and invite conflicts when those branches later merge.
- Pick small, self-contained commits. A commit that depends on earlier ones will conflict or break the build.
- Use merge or rebase to move whole features; cherry-pick is for surgical moves.

In the interview: "A payment-timeout fix was merged to `main` during a release freeze. I cherry-picked it with `-x` onto `release/2.3`, ran the release pipeline and tagged 2.3.1, so the audit trail showed where the fix came from."

Q2. What is git checkout?

`git checkout` moves `HEAD` and updates your working tree, and it does two jobs: switching branches (or commits) and restoring files. Since Git 2.23 those jobs are split into `git switch` and `git restore`, which are clearer and safer; `checkout` still works everywhere.

```

# Branches
git checkout feature/login           # switch: moves HEAD, updates index
and working tree
git checkout -b feature/payment      # create from current HEAD and switch
git checkout -b hotfix/2.3.1 v2.3.0  # create from a tag
git checkout -t origin/develop       # local branch that tracks the remote
one
git checkout -                       # back to the previous branch

# Files
git checkout -- app/config.yaml      # discard unstaged changes to one
file
git checkout HEAD~2 -- app/config.yaml # the file as it was two commits ago
git checkout develop -- Jenkinsfile  # take one file from another branch

# A specific commit (detached HEAD)
git checkout 3f2a9c1

```

Older form	Git 2.23+ form
<code>git checkout feature/login</code>	<code>git switch feature/login</code>
<code>git checkout -b feature/payment</code>	<code>git switch -c feature/payment</code>
<code>git checkout -- file</code>	<code>git restore file</code>
<code>git checkout HEAD~2 -- file</code>	<code>git restore --source=HEAD~2 file</code>
<code>git reset HEAD file (unstage)</code>	<code>git restore --staged file</code>

Detached HEAD: checking out a commit or tag points `HEAD` at that commit instead of at a branch. Commits made there belong to no branch and are eventually garbage-collected, so keep them with `git switch -c <new-branch>`. CI tools such as Jenkins and GitHub Actions check out the exact commit SHA, so a detached HEAD inside a pipeline is normal.

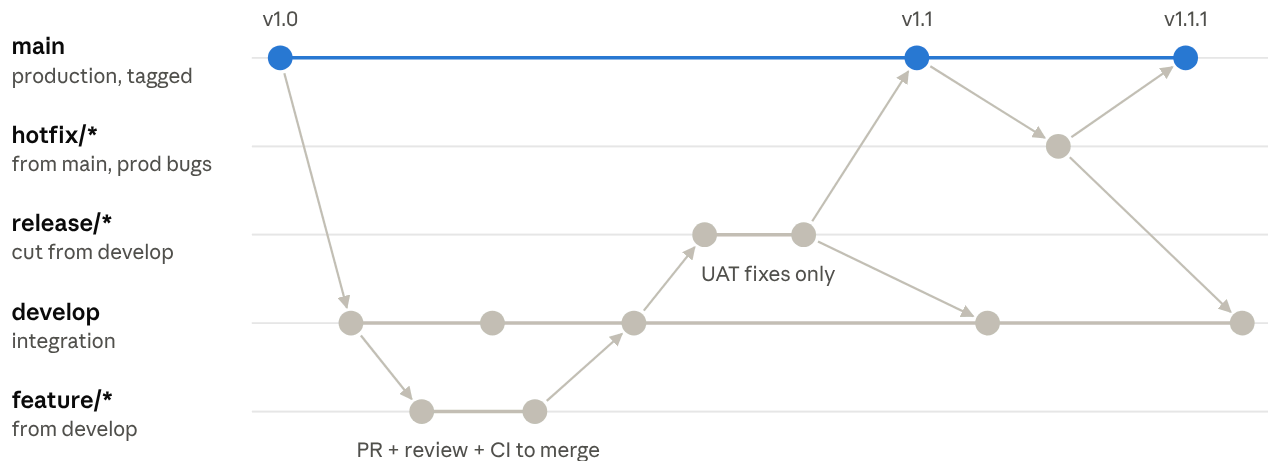
When the switch is refused: Git will not switch if uncommitted changes would be overwritten. Commit them, `git stash` them, or use `git worktree add ../hotfix hotfix/2.3.1` to work on two branches side by side.

Q3. Explain your git branching strategy

A branching strategy is the team's agreement on which branches exist, how long they live, how code moves between them and which branch deploys where. Compare the three common models, then explain the one you ran and why it fit.

In GitFlow only release and hotfix branches reach main

Time runs left to right; each arrow is a branch-off or a merge



GitFlow · 5 branch types, branch-offs and merges

Features meet in `develop` through pull requests; only `release/*` and `hotfix/*` merge into `main`, and every merge to `main` is tagged.

GitFlow (planned releases, several versions in support):

- `main`: production code only; every merge is tagged (`v2.4.0`).
- `develop`: integration branch for the next release.
- `feature/*`: branched from `develop`, merged back by pull request.
- `release/*`: cut from `develop` for stabilization (UAT fixes only), merged to `main` and back to `develop`.
- `hotfix/*`: branched from `main` for production bugs, merged to `main` and `develop`.

GitHub Flow (one version, continuous delivery): `main` is always deployable. Short-lived branch → pull request → green CI and review → merge → deploy.

Trunk-based development (high-frequency teams): everyone merges small changes to `main` at least daily. Branches live hours, not weeks; unfinished work hides behind feature flags; releases are tags or short release branches.

	GitFlow	GitHub Flow	Trunk-based
Long-lived branches	<code>main</code> , <code>develop</code>	<code>main</code>	<code>main</code>
Release cadence	Planned, every few weeks	Every merge	Many per day
Merge pain	High	Low	Lowest
Prerequisites	Release manager, UAT cycle	Solid CI, fast reviews	Strong automated tests, feature flags
Best fit	Client releases, several versions live	SaaS web apps	Mature CI/CD, microservices

Controls that make any strategy work:

- **Branch protection** on `main` and `develop`: no direct pushes; pull request with one or two approvals; required checks (build, unit tests, SonarQube quality gate, image scan); branch up to date before merge.
- **CODEOWNERS** for mandatory reviewers on sensitive paths such as `/helm/` and `/terraform/`.
- **Naming**: `feature/JIRA-123-short-desc` , `bugfix/...` , `hotfix/2.3.1` , `release/2.4.`
- **Versioning**: semantic-version tags on `main` trigger the production pipeline; Conventional Commits (`feat:` , `fix:`) can generate the changelog.
- **Branch-to-environment map**: `feature` → `dev`, `develop` → QA, `release/*` → UAT, tag on `main` → production.

In the interview: "We ran a trimmed GitFlow because the client signed off every release in UAT. Features reached `develop` by pull request with two approvals and a SonarQube gate; `release/x.y` went to UAT; a tag on `main` triggered production. Hotfixes branched from `main` and were merged back to `develop` the same day. Newer microservices moved to trunk-based with feature flags, which cut merge conflicts and lead time."

Q4. What is a git merge conflict?

A merge conflict happens when Git cannot combine two lines of history automatically, so it stops and asks you to decide. The same thing can happen during merge, rebase, cherry-pick, revert and `git stash pop`.

Common causes:

- The same lines of the same file changed differently on both branches.
- One branch modified a file that the other deleted or renamed (modify/delete conflict).
- Both branches added a file at the same path, or both changed a binary file.

What you see:

```
$ git merge feature/perf
Auto-merging helm/values.yaml
CONFLICT (content): Merge conflict in helm/values.yaml
Automatic merge failed; fix conflicts and then commit the result.
```

```
replicaCount: 3
<<<<<<< HEAD
timeoutSeconds: 30      # current branch (main)
=====
timeoutSeconds: 60      # incoming branch (feature/perf)
>>>>>>> feature/perf
```

How to resolve:

```
git status                                # conflicted files show as "both
modified"
git diff --name-only --diff-filter=U      # only the conflicted files
# edit each file: keep the correct result, delete the <<<<<<< ===== >>>>>>>
markers
git add helm/values.yaml
git commit                                # or: git merge --continue
git merge --abort                          # give up and return to the pre-
merge state

git checkout --ours path/file              # keep your side for a whole file
git checkout --theirs path/file            # keep the incoming side
git mergetool                             # or the 3-way merge editor in VS
Code / IntelliJ
git log --merge -p path/file               # commits on both sides that
touched the file
```

During a rebase, "ours" is the branch you are rebasing onto and "theirs" is your own commit, the reverse of a merge. Continue with `git rebase --continue`, or use `--skip` or `--abort`.

Prevention:

- Keep branches small and short-lived; merge or rebase from `main` daily.
- Split large shared files: one Helm values file per environment, modular Terraform.
- Run formatters and linters in pre-commit hooks so whitespace changes do not collide.
- Enable `git config --global rerere.enabled true` so Git reuses resolutions it has already recorded.
- After resolving, build and run the tests: a clean text merge can still produce a broken program.

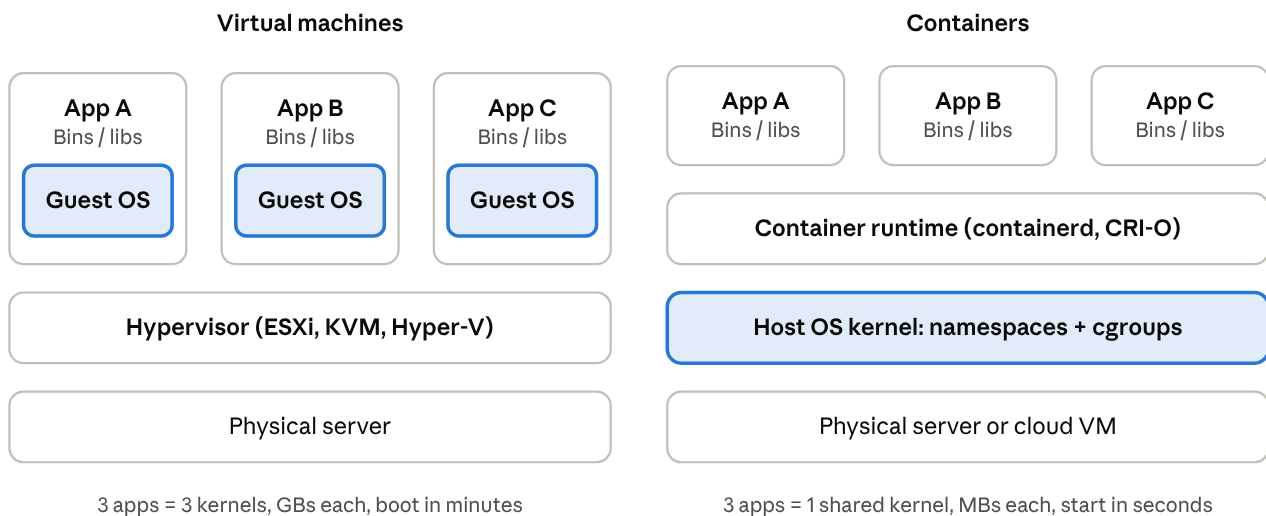
Part 2 · Docker and containerization

Q5. Virtualization vs containerization

Virtualization splits physical hardware into virtual machines, each with its own guest OS kernel; containerization splits one OS kernel into isolated processes. VMs give stronger isolation; containers give speed, density and portability.

Each VM carries its own kernel; containers share one

Highlighted boxes are operating-system kernels



VM stack vs container stack · kernels highlighted

Three VMs boot three guest kernels on a hypervisor; three containers share the host's one kernel and are isolated by namespaces and cgroups.

How each one isolates:

- **Virtual machines:** a hypervisor emulates hardware. Type 1 runs on bare metal (VMware ESXi, Hyper-V, KVM, Xen); Type 2 runs on a host OS (VirtualBox, VMware Workstation).
- **Containers:** the shared Linux kernel isolates processes with namespaces (pid, net, mnt, uts, ipc, user) and caps their resources with cgroups (CPU, memory, I/O). The image is a stack of read-only layers plus one writable layer (overlay2).

	Virtual machine	Container
What it packages	Full guest OS + app	App + its libraries only
Kernel	One per VM	Shared with the host
Typical size	GBs	MBs
Start time	Minutes	Seconds or less
Isolation	Strong, hardware level	Process level; a kernel flaw affects every container
Density per host	Tens	Hundreds
Portability	Tied to the hypervisor's image format	Same image on a laptop, in CI, on any cloud
OS mix	Any guest OS on any host	Linux containers need a Linux kernel
Best for	Legacy apps, mixed OSes, hard multi-tenancy	Microservices, CI/CD, autoscaling

Senior points:

- They are complementary: on EKS, AKS and GKE the containers run on worker nodes that are themselves VMs.
- When container isolation is not enough, sandboxed runtimes close the gap: gVisor (a user-space kernel), Kata Containers and Firecracker (a lightweight microVM per pod).
- Docker Desktop on Windows and macOS runs a small Linux VM underneath, which is why Linux containers work there.

Q6. What is a Dockerfile?

A Dockerfile is a text file of instructions that `docker build` runs top to bottom to produce an image. Instructions that change the filesystem (`RUN` , `COPY` , `ADD`) create layers, and the build cache reuses each layer until that instruction or its inputs change.

Production example, a multi-stage build for a Spring Boot service:

```
# ----- Stage 1: build -----
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /src
# copy the pom first: the dependency layer stays cached until pom.xml changes
COPY pom.xml .
RUN mvn -B dependency:go-offline
COPY src ./src
RUN mvn -B package -DskipTests

# ----- Stage 2: runtime -----
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN addgroup -S app && adduser -S app -G app
COPY --from=build /src/target/*.jar app.jar
# never run as root
USER app
# documentation only: -p on docker run publishes the port
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s --retries=3 CMD wget -qO-
http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "-jar", "app.jar"]
```

Maven and the JDK stay in the build stage; the runtime image holds only a JRE and one jar, so it is much smaller and carries far fewer CVEs. Note that Dockerfile comments must sit on their own line.

```
docker build -t registry.example.com/orders-api:1.4.2 .
docker run -d -p 8080:8080 --name orders registry.example.com/orders-
api:1.4.2
docker history registry.example.com/orders-api:1.4.2    # layers and their
sizes
```

Pairs interviewers probe:

Pair	Difference
<code>CMD</code> vs <code>ENTRYPOINT</code>	<code>ENTRYPOINT</code> is the executable; <code>CMD</code> supplies default arguments, replaced by anything after the image name in <code>docker run</code> .
Exec vs shell form	<code>["java", "-jar", "app.jar"]</code> runs Java as PID 1, so it receives <code>SIGTERM</code> ; <code>java -jar app.jar</code> runs under <code>/bin/sh -c</code> , which may not forward signals.
<code>COPY</code> vs <code>ADD</code>	<code>ADD</code> also unpacks local tar files and fetches URLs; use <code>COPY</code> unless you need that.
<code>ARG</code> vs <code>ENV</code>	<code>ARG</code> exists only during the build; <code>ENV</code> persists into the running container.
<code>EXPOSE</code> vs <code>-p</code>	<code>EXPOSE</code> documents the port; <code>-p 8080:8080</code> actually publishes it.

Best practices:

- Small, pinned base images (`-alpine` , `-slim` , distroless or Red Hat UBI); pin by digest in production.
- Order layers from least to most frequently changed: dependencies before source code.
- Install packages and clean caches in the same `RUN` (`apt-get install ... && rm -rf /var/lib/apt/lists/*`).
- A `.dockerignore` for `.git` , `target/` , `node_modules/` and local `.env` files.
- No secrets in the image or in `ARG` ; use BuildKit secret mounts, e.g. `RUN --mount=type=secret,id=npnrc,target=/root/.npmrc npm ci` .
- Non-root `USER` , read-only root filesystem at runtime, one main process per container.
- Scan in CI (Trivy, Gype) and fail the build on critical CVEs.
- Kubernetes ignores `HEALTHCHECK` ; it uses liveness and readiness probes instead (Q9).

Q7. What is Docker Compose?

Docker Compose defines and runs a multi-container application from one YAML file: its services, networks, volumes and configuration. One command, `docker compose up` , builds and starts the whole stack on a single host.

Where it fits: local development environments, integration tests in CI and small single-server deployments. It is not a multi-node orchestrator; production clusters use Kubernetes. Compose V2 is the `docker compose` CLI plugin, the old Python `docker-compose` (V1) is retired, and the top-level `version:` key is obsolete.

Example: Nginx in front of an API, with Postgres and Redis.

```
# compose.yaml
services:
  web:
    image: nginx:1.27-alpine
    ports:
      - '80:80' # host:container
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - api

  api:
    build: ./api # builds ./api/Dockerfile
    image: myorg/orders-api:1.4.2
    environment:
      DB_HOST: db # service name = DNS name on the
project network
      DB_NAME: orders
      DB_USER: app
      DB_PASSWORD: ${DB_PASSWORD} # from a .env file beside
compose.yaml, never committed
      REDIS_HOST: cache
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_started
    restart: unless-stopped

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: orders
      POSTGRES_USER: app
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
```

```

    - pgdata:/var/lib/postgresql/data # named volume survives 'down'
healthcheck:
  test: ['CMD-SHELL', 'pg_isready -U app -d orders']
  interval: 10s
  timeout: 5s
  retries: 5

cache:
  image: redis:7-alpine

volumes:
  pgdata:

```

Compose creates a project network (`<project>_default`), so `api` reaches Postgres at `db:5432` by name. Only `web` publishes a port to the host; the database is unreachable from outside.

```

docker compose up -d                # create and start everything
docker compose up -d --build api    # rebuild and restart one service
docker compose ps
docker compose logs -f api
docker compose exec db psql -U app -d orders
docker compose up -d --scale api=3  # works because api publishes no
fixed host port
docker compose config                # validate and print the merged file
docker compose -f compose.yaml -f compose.prod.yaml up -d # layered
overrides
docker compose down                 # remove containers and the network
docker compose down -v              # also delete named volumes: data
loss

```

Useful extras: `compose.override.yaml` is merged automatically for local tweaks, `profiles:` keeps optional services (debug tools, mocks) off unless `--profile` names them, and `env_file:` loads variables per service.

Q8. What is `depends_on` in Docker Compose?

`depends_on` sets start-up and shutdown order between services. In its short form it waits only until the dependency's container has started, not until the application inside it is ready.

```
depends_on:          # short form: start order only
  - db
  - cache
```

The long form adds a condition:

Condition	Compose waits until the dependency
<code>service_started</code> (default)	has a running container
<code>service_healthy</code>	reports healthy through its <code>healthcheck</code>
<code>service_completed_successfully</code>	has run and exited with code 0 (one-off jobs)

Example: run database migrations once, then start the API.

```
services:
  db:
    image: postgres:16-alpine
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U app']
      interval: 5s
      retries: 10

  migrate:
    image: myorg/orders-api:1.4.2
    command: ['./migrate.sh']          # e.g. Flyway or Liquibase
    depends_on:
      db:
        condition: service_healthy

  api:
    image: myorg/orders-api:1.4.2
    depends_on:
      db:
        condition: service_healthy
        restart: true                  # restart api when Compose restarts
db
  migrate:
    condition: service_completed_successfully
```

Points to make:

- `docker compose up api` also starts its dependencies; `--no-deps` skips them. `docker compose down` stops services in reverse order.
- `depends_on` only orders start-up. If Postgres restarts an hour later, the app must still reconnect, so build retries with backoff into the code.
- `required: false` makes a dependency optional (Compose 2.20 and later).
- Kubernetes has no `depends_on`. The equivalents are init containers (wait for the DB, run migrations), readiness probes, a Helm pre-install hook or Argo CD sync wave for migrations, and retry logic in the app.

Q9. Can a container restart itself?

No: the process inside a container can only exit. What restarts it is the supervisor around it: the Docker daemon (restart policy), the kubelet in Kubernetes, or systemd. The application's job is to fail fast, by exiting non-zero or failing a health check, so the supervisor can act.

Docker restart policies:

Policy	Restarts the container when	When the Docker daemon restarts
<code>no</code> (default)	never	stays stopped
<code>on-failure[:N]</code>	it exits non-zero, optionally at most N times	not restarted
<code>always</code>	it stops for any reason	started again, even if you had stopped it manually
<code>unless-stopped</code>	it stops for any reason	started again, unless you had stopped it manually

A manual `docker stop` is always respected until the daemon restarts. Docker waits longer between each attempt (100 ms, doubling, capped at one minute) and resets the delay once the container stays up for 10 seconds.

```
docker run -d --restart=on-failure:5 myorg/orders-api:1.4.2
docker run -d --restart=unless-stopped nginx:1.27-alpine
docker update --restart=always orders          # change the policy on an
existing container
docker inspect -f '{{.RestartCount}} {{.HostConfig.RestartPolicy.Name}}'
orders
```

In Compose the same setting is `restart: unless-stopped`. The gap to mention: plain Docker restarts on exit, not on health. A container stuck as `unhealthy` keeps running; Swarm or an autoheal helper is needed to act on it, while Kubernetes handles it natively with liveness probes.

Kubernetes:

- `spec.restartPolicy` is `Always` (the default, and the only option for Deployments), `OnFailure` or `Never` (for Jobs).
- The kubelet restarts the container in place: same pod, same node, same IP. Delays back off exponentially (10 s, 20 s, 40 s, capped at 5 minutes), shown as `CrashLoopBackOff`; the delay resets after 10 minutes of healthy running.
- A failed **liveness** probe makes the kubelet kill and restart the container.
- A failed **readiness** probe restarts nothing; the pod is taken out of Service endpoints until it passes again.
- A **startup** probe holds off liveness checks while a slow application boots.
- If the pod itself is deleted or its node dies, that is rescheduling, not a restart: the ReplicaSet creates a new pod with a new name and IP.

```
containers:
- name: api
  image: myorg/orders-api:1.4.2
  startupProbe:
    # up to 30 x 5 s = 150 s to boot
    httpGet: { path: /actuator/health, port: 8080 }
    failureThreshold: 30
    periodSeconds: 5
  livenessProbe:
    # hung process -> restart
    httpGet: { path: /actuator/health/liveness, port: 8080 }
    periodSeconds: 10
    failureThreshold: 3
  readinessProbe:
    # not ready -> no traffic, no restart
    httpGet: { path: /actuator/health/readiness, port: 8080 }
    periodSeconds: 5
```

Keep liveness checks local to the process. If liveness also checks the database, one database outage restarts every pod at once.

```

kubectl get pod -w                                # RESTARTS column,
CrashLoopBackOff
kubectl describe pod <pod>                        # Last State, Reason, Exit Code;
probe failures in Events
kubectl logs <pod> -c api --previous              # logs from the crashed instance

```

Exit codes to know: 137 is SIGKILL, often OOMKilled (raise the memory limit or fix the leak); 143 is SIGTERM; 1 is an application error; 139 is a segmentation fault.

Q10. What is your strategy to containerize an application?

Work in a fixed order: understand the app, make it fit containers (stateless, configurable, observable), then build, secure, ship and run the image. The goal is one immutable image promoted unchanged from dev to production, with only configuration changing per environment.

1. **Assess the app.** Runtime and version, OS packages, ports, start and stop commands, config files, external dependencies, where it writes data, how it logs, licensing. Check it against the 12-factor principles.
2. **Externalize configuration.** Environment variables, ConfigMaps and Secrets instead of per-environment builds; no hard-coded IPs or hostnames.
3. **Remove local state.** Sessions to Redis, uploads to object storage (S3, Azure Blob), scheduled jobs to Kubernetes CronJobs, databases to a managed service or a StatefulSet with persistent volumes.
4. **Make it observable.** Logs to stdout and stderr, health endpoints for liveness and readiness, a metrics endpoint for Prometheus.
5. **Write the Dockerfile.** Multi-stage, minimal pinned base image, non-root user, exec-form ENTRYPOINT, graceful SIGTERM handling (Q6).
6. **Run it locally with Compose** next to its dependencies (Q7) and run the test suite against the container.
7. **Secure the supply chain.** Image scan (Trivy), SBOM (Syft), signing (Cosign), scheduled base-image updates.
8. **Automate in CI.** Build → unit tests → image build → scan → push to the registry (ECR, ACR, Harbor, Nexus), tagged with the git SHA and semantic version; tags are immutable.
9. **Deploy to Kubernetes.** Helm chart or Kustomize with requests and limits, probes, HPA, PodDisruptionBudget and anti-affinity; deliver through Argo CD or the pipeline.

10. Tune and harden. Load test, size the JVM heap to the container limit, choose the rollout (rolling, blue/green or canary) and rehearse the rollback.

Blockers you will meet, and the usual fix:

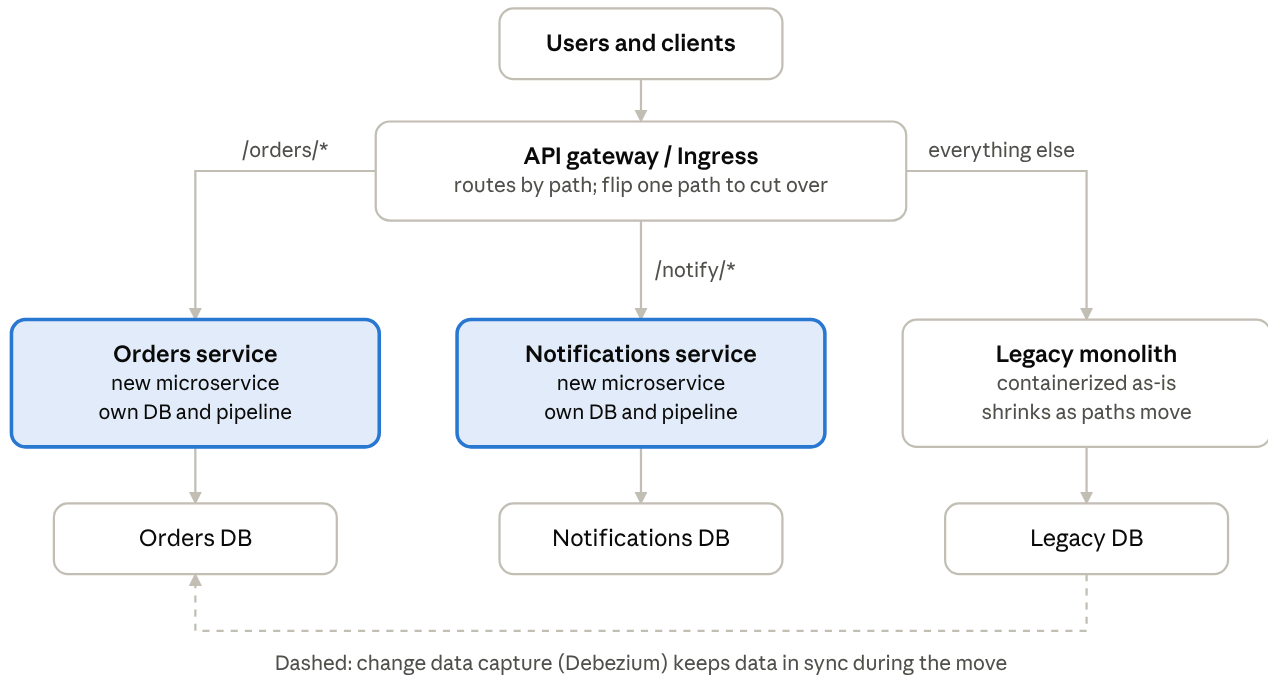
Blocker	Fix
App writes logs to files	Log to stdout, or a sidecar ships the file
Sessions held in memory, sticky sessions	Move sessions to Redis
A separate build per environment	One image, configuration injected at run time
Licence tied to host name or MAC address	Raise it with the vendor early; it may stay on a VM
Long start-up time	Startup probe, warm-up, lazy initialization
Needs root or a privileged port	Listen on 8080 or above, map ports, drop Linux capabilities

Q11. How would you migrate a monolith to containers?

In two moves: first containerize the monolith as it is (replatform) and run it on the new platform, then break it into services one slice at a time with the strangler fig pattern. A big-bang rewrite is the classic failure; every phase below is releasable and reversible.

The gateway moves one path at a time from the monolith to new services

Highlighted: new services; the monolith keeps every path not yet moved



Strangler fig migration · gateway, 2 new services, the monolith

Each path flipped at the gateway moves one capability out of the monolith; when no path is left, the monolith and its database are retired.

Phase 1: Discover. Map modules, call paths, data ownership, integrations, batch jobs and SLAs (RPO/RTO). Pick candidate service boundaries with domain-driven design. AWS App2Container, Azure Migrate or Konveyor (Move2Kube) can generate starter Dockerfiles and manifests for Java and .NET apps.

Phase 2: Build the platform. A Kubernetes cluster (EKS, AKS or OpenShift) built with Terraform, plus a registry, CI/CD, GitOps (Argo CD), secrets management, monitoring and logging, and network and IAM rules.

Phase 3: Lift the monolith into a container. One image, configuration externalized, sessions in Redis, files in object storage, logs to stdout. Run several replicas behind Ingress and prove parity with functional and performance tests against the old VMs.

Phase 4: Strangle it. Put an API gateway or Ingress in front and move one bounded context at a time, starting with something low-risk and loosely coupled (notifications, reporting, authentication). Route its paths (`/orders/*`) to the new service while

everything else still reaches the monolith. Give each new service its own database, keep data in sync during the move with change data capture (Debezium) or events, and use sagas instead of distributed transactions.

Phase 5: Cut over and retire. Canary or blue/green with weighted routing, feature flags and a rehearsed rollback. Once a module receives no traffic, delete its code from the monolith; when nothing is left, decommission the VMs.

Risks to name: a shared database that keeps services coupled, chatty calls between new services, data consistency during dual writes, team skills, and splitting too finely too early.

In the interview (STAR): "Situation: a Java order-management monolith on four Tomcat VMs, released every six weeks with downtime. **Task:** move to Kubernetes without a release freeze. **Action:** containerized it as-is with a multi-stage Dockerfile, moved sessions to Redis and uploads to S3, deployed with Helm on EKS behind an ingress, then carved out notifications and reporting as services routed by path. **Result:** zero-downtime rolling releases, weekly deploys, and the VMs decommissioned." Replace every detail with your own project's.

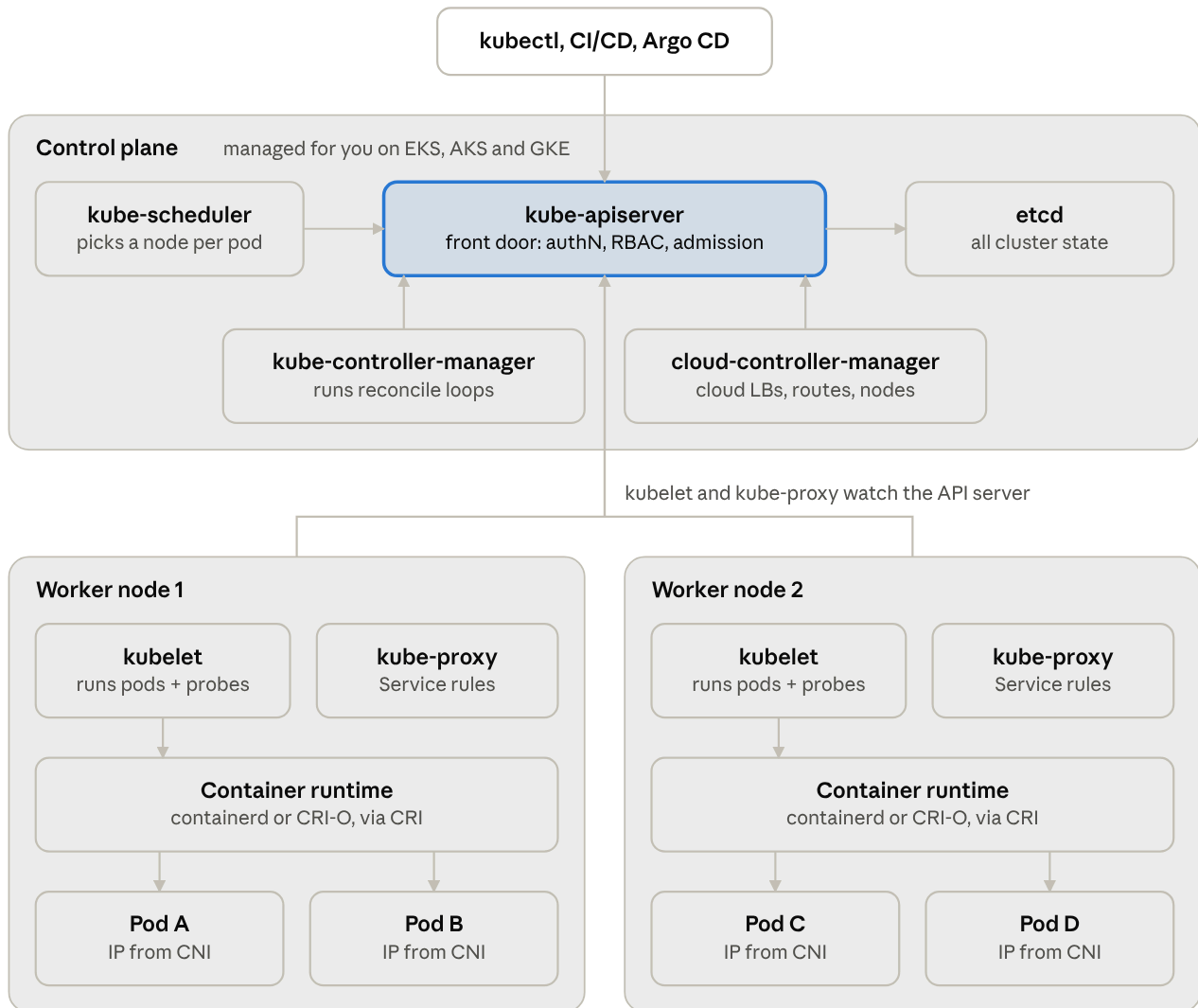
Part 3 · Kubernetes

Q12. Explain Kubernetes architecture

A cluster has a control plane that stores the desired state and makes decisions, and worker nodes that run the pods. Every component talks only to the API server, and controllers keep reconciling the actual state toward the desired state.

Every Kubernetes component works through the API server

Highlighted: kube-apiserver, the only component that reads and writes etcd



Kubernetes architecture · control plane and 2 worker nodes

Read it top-down: requests enter through the API server, which alone touches etcd; the scheduler, the controllers and every node agent watch the API server and act on what changes.

Control plane:

Component	Job
<code>kube-apiserver</code>	Front door for every request (kubectl, CI, other components). Authenticates, authorizes, runs admission, validates and writes to etcd. The only component that talks to etcd; scales horizontally.
<code>etcd</code>	Consistent key-value store (Raft) holding all cluster state. Run 3 or 5 members and back it up (<code>etcdctl snapshot save</code>).
<code>kube-scheduler</code>	Picks a node for each unscheduled pod: filters nodes (resources, taints and tolerations, node selectors, affinity), then scores the rest.
<code>kube-controller-manager</code>	Runs the control loops: Deployment, ReplicaSet, Node, Job, EndpointSlice, ServiceAccount and more.
<code>cloud-controller-manager</code>	Cloud integration: load balancers for <code>LoadBalancer</code> Services, routes, node lifecycle.

Worker node:

Component	Job
<code>kubelet</code>	Agent on every node. Watches for pods assigned to it, starts containers through the runtime (CRI), runs probes, mounts volumes, reports status.
<code>kube-proxy</code>	Programs iptables, IPVS or nftables rules so Service IPs reach pod IPs (Q17).
Container runtime	containerd or CRI-O. Docker Engine support (dockershim) was removed in v1.24; images built with Docker still run.
CNI plugin	Pod networking and pod IPs: Calico, Cilium, Flannel, AWS VPC CNI, Azure CNI.

Add-ons: CoreDNS for service discovery, an Ingress or Gateway controller, metrics-server for autoscaling, CSI drivers for storage.

What happens on `kubectl apply -f deployment.yaml` :

1. kubectl sends the Deployment to the API server, which authenticates the caller, checks RBAC, runs admission controllers and stores the object in etcd.
2. The Deployment controller creates a ReplicaSet; the ReplicaSet controller creates pod objects that have no node yet.
3. The scheduler picks a node for each pod and records it in `spec.nodeName`.
4. The kubelet on that node pulls the image through containerd, the CNI plugin gives the pod an IP, and the containers start.
5. When the readiness probe passes, the pod IP joins the Service's EndpointSlice and kube-proxy updates the rules on every node.

High availability: three control-plane nodes behind a load balancer, etcd stacked on them or run separately, and worker nodes spread across availability zones. On EKS, AKS and GKE the provider runs the control plane; you own the nodes, add-ons and workloads.

In the interview: practise drawing this from memory: control plane on top, nodes below, every arrow ending at the API server. Then narrate the five `kubectl apply` steps over the drawing.

Q13. What is a LoadBalancer in Kubernetes?

A Service of `type: LoadBalancer` asks the cloud to create an external Layer-4 load balancer with its own IP that forwards to the Service's pods. It builds on the other types: the Service still gets a ClusterIP and a NodePort.

Service type	Reachable from	Typical use
<code>ClusterIP</code> (default)	Inside the cluster only	Service-to-service calls
<code>NodePort</code>	<code><any node IP>:30000-32767</code>	Testing, or behind your own external LB
<code>LoadBalancer</code>	An external IP from the cloud LB	Exposing one TCP or UDP service
<code>ExternalName</code>	DNS CNAME to an outside name	Pointing at an external database or API
Headless (<code>clusterIP: None</code>)	DNS returns the pod IPs	StatefulSets, client-side load balancing

```

apiVersion: v1
kind: Service
metadata:
  name: orders-api
  annotations:                                # AWS Load Balancer Controller ->
Network Load Balancer
  service.beta.kubernetes.io/aws-load-balancer-type: external
  service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: ip
  service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing
spec:
  type: LoadBalancer
  selector:
    app: orders-api
  ports:
    - name: http
      port: 80                                # LB and Service port
      targetPort: 8080                        # container port
  externalTrafficPolicy: Local                # keep the client source IP, skip the
extra hop

```

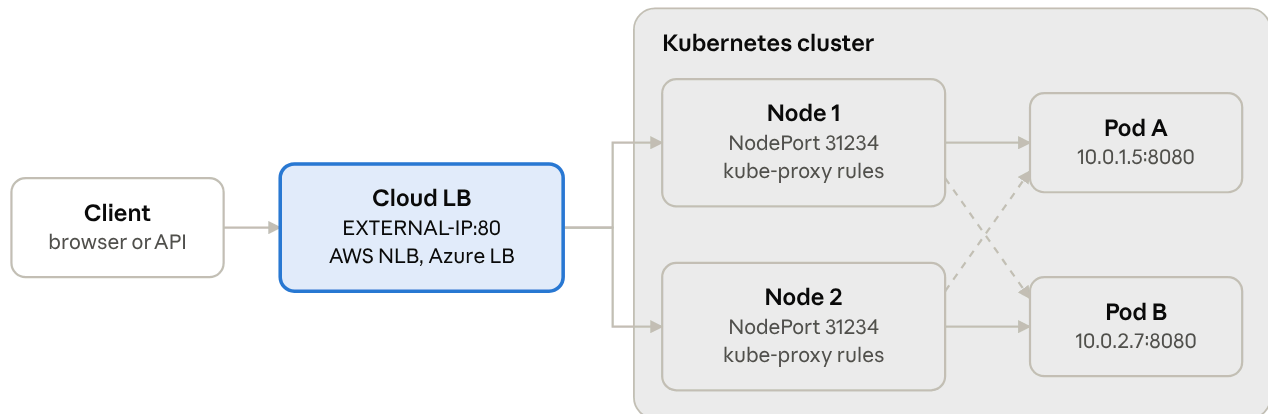
```

kubectl get svc orders-api
# NAME          TYPE          CLUSTER-IP    EXTERNAL-IP
PORT(S)
# orders-api    LoadBalancer 10.100.23.17   k8s-prod-ordersap-xxxx.elb.ap-
south-1.amazonaws.com 80:31234/TCP

```

A LoadBalancer Service puts a cloud LB in front of the NodePorts

Highlighted: what the cloud provider adds for this Service type



Dashed: with externalTrafficPolicy: Cluster, a node may forward to a pod on another node; Local avoids that hop

The Service keeps its ClusterIP for in-cluster callers: 10.100.23.17:80 to targetPort 8080

Service type LoadBalancer · traffic path from client to pods

The cloud LB spreads traffic over every node's NodePort, and the kube-proxy rules on each node pick a ready pod, possibly on another node.

How it works: the cloud-controller-manager (or the AWS Load Balancer Controller) sees the Service and creates the load balancer. The LB sends traffic to the NodePort on each node (instance mode) or straight to pod IPs (IP mode), and kube-proxy rules pick a ready pod.

`externalTrafficPolicy`:

- `Cluster` (default): every node accepts traffic and may forward it to a pod on another node; the client IP is lost to source NAT.
- `Local`: only nodes running a ready pod receive traffic; the client IP is preserved, but load can spread unevenly.

For internal-only services use the provider's internal annotation, such as `aws-load-balancer-scheme: internal` on AWS or `service.beta.kubernetes.io/azure-load-balancer-internal: "true"` on Azure.

On-premises clusters have no cloud API, so the external IP stays `<pending>`. Install MetalLB (Layer 2 or BGP mode), kube-vip or OpenELB to hand out addresses from a pool you own (Q18). This matters for enterprises that run their own data centres.

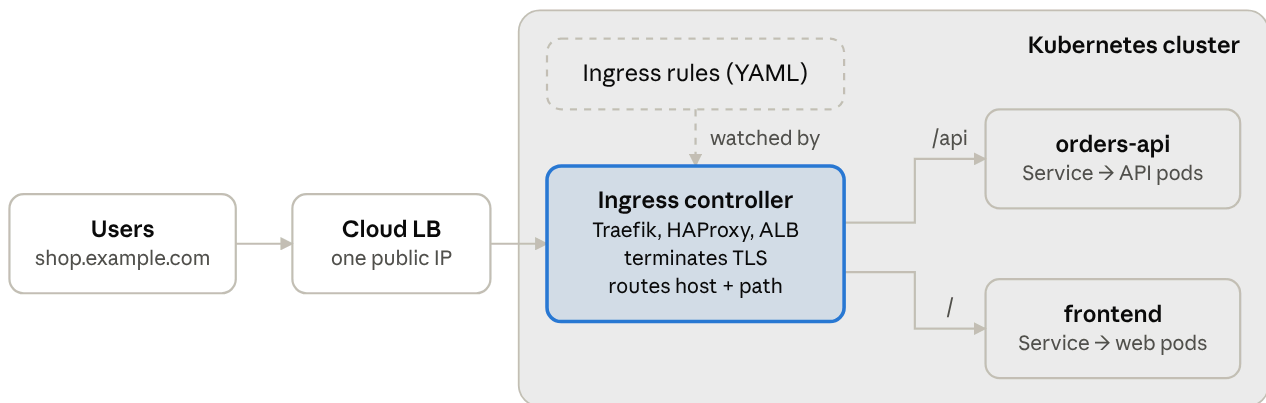
Limits: one cloud LB per Service gets expensive, and Layer 4 means no host or path routing and no central TLS. For HTTP apps, put one LoadBalancer in front of an Ingress or Gateway controller instead (Q14).

Q14. What is an Ingress controller?

An Ingress is a Kubernetes object holding Layer-7 routing rules (host and path → Service, plus TLS). An Ingress controller is the running reverse proxy that watches those objects and applies them; without a controller, an Ingress does nothing.

One cloud LB serves every app; the Ingress controller routes by host and path

Highlighted: the controller, a reverse proxy running as pods in the cluster



Without Ingress every Service needs its own cloud LB; with it: one LB, central TLS, host and path routing

Ingress · one cloud LB, the controller, routes to 2 Services

The cloud LB forwards all traffic to the controller pods, which terminate TLS and route `/api` to orders-api and `/` to the frontend, following the Ingress rules they watch.

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  namespace: prod
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod # cert-manager issues
the TLS certificate
spec:
  ingressClassName: traefik # which controller handles this Ingress
  tls:
  - hosts: [shop.example.com]
    secretName: shop-tls

```

```

rules:
- host: shop.example.com
  http:
    paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: orders-api
            port:
              number: 80
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend
            port:
              number: 80

```

Common controllers: Traefik, HAProxy, F5 NGINX Ingress Controller, Kong, AWS Load Balancer Controller (provisions an ALB), Azure Application Gateway Ingress Controller, GKE Ingress, Istio and Contour.

Current-state point worth raising: the community `ingress-nginx` controller, long the default choice, was retired in March 2026. It gets no further releases, bug fixes or security patches; existing installs keep working, and the project recommends moving to Gateway API or another controller ([Kubernetes announcement](#)). The [Ingress2Gateway](#) tool helps convert existing Ingress objects.

	Service LoadBalancer	Ingress + controller
OSI layer	L4 (TCP/UDP)	L7 (HTTP/HTTPS, gRPC)
Routing	One IP → one Service	Host and path → many Services
TLS	Per service, if at all	Terminated centrally, certificates from cert-manager
Cost	One cloud LB per Service	One cloud LB for all apps
Extras	None	Rewrites, rate limits, auth, canary by header or weight (controller-specific)

Gateway API, the successor to Ingress, splits the job by role: **GatewayClass** (which implements), **Gateway** (listeners and certificates, owned by the platform team) and **HTTPRoute** (routes, owned by app teams). It standardizes what Ingress left to vendor annotations, such as traffic splitting and header matching.

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: orders
  namespace: prod
spec:
  parentRefs:
  - name: public-gateway
    namespace: infra
  hostnames: [shop.example.com]
  rules:
  - matches:
    - path: { type: PathPrefix, value: /api }
    backendRefs:
      # 90/10 canary, no annotations needed
    - name: orders-api
      port: 80
      weight: 90
    - name: orders-api-v2
      port: 80
      weight: 10
```

Troubleshooting a 404 or 502: `kubectl describe ingress shop`, then check `ingressClassName`, the Service name and port, ready endpoints (`kubectl get endpointslices -l kubernetes.io/service-name=orders-api`) and the controller's pod logs.

Q15. Can we delete a pod? Can multiple containers run in one pod?

Yes to both. Deleting a pod is routine, and what happens next depends on what owns it; a pod can also run several containers that share one network namespace and its volumes.

Deleting a pod:

```

kubectl delete pod orders-api-7d9c5b6f4-x2k8p
kubectl delete pod <pod> --grace-period=0 --force    # only for pods stuck in
Terminating; risky for StatefulSets
kubectl rollout restart deployment/orders-api        # the right way to
restart every pod
kubectl scale deployment/orders-api --replicas=0    # stop all pods but keep
the Deployment

```

Owner	After kubectl delete pod
Deployment / ReplicaSet	A replacement is created at once: new name, new IP, possibly another node
StatefulSet	Recreated with the same name (<code>mysql-0</code>) and the same PVC
DaemonSet	Recreated on the same node
Job	Replaced if the Job has not completed yet
None (a bare pod)	Gone for good

Graceful shutdown runs in this order:

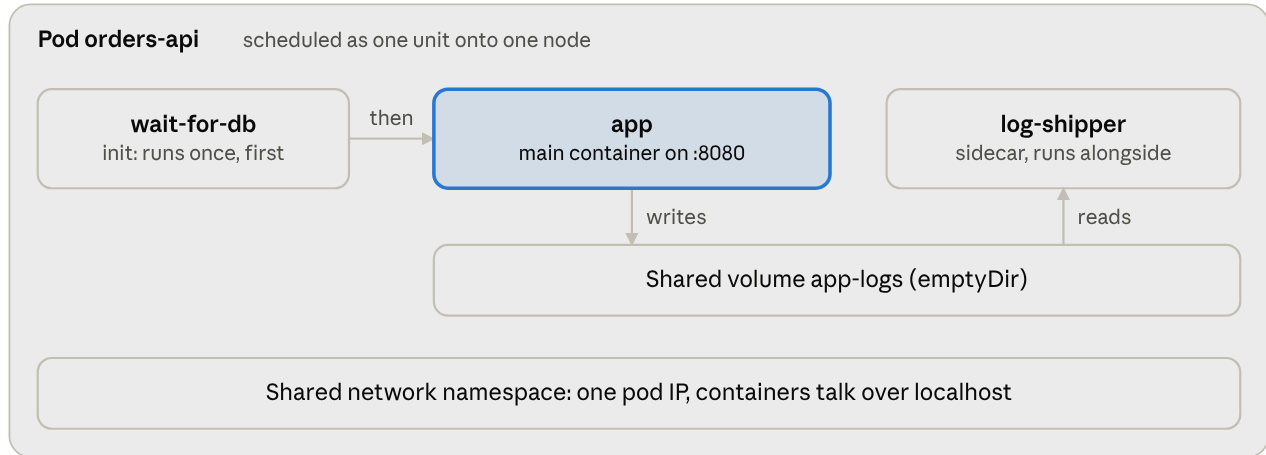
1. The pod is marked `Terminating` and removed from Service endpoints.
2. The `preStop` hook runs, for example `sleep 5` so load balancers stop sending traffic.
3. SIGTERM goes to PID 1 of each container.
4. After `terminationGracePeriodSeconds` (default 30) anything still running gets SIGKILL.

A `PodDisruptionBudget` limits evictions (`kubectl drain` , the cluster autoscaler) but not a direct `kubectl delete pod` . A pod stuck in `Terminating` is usually waiting on a finalizer or on an unreachable node.

Multiple containers in one pod: they are scheduled together onto one node, share the pod IP (they talk over `localhost` and cannot both use one port), can share volumes, and start and stop as a unit.

Containers in one pod share an IP and can share volumes

Highlighted: the main app container



Native sidecar: an init container with `restartPolicy: Always` starts before the app and stops after it

Multi-container pod · init container, app, sidecar, shared volume

The init container must finish before the app starts; the app and its sidecar then run side by side, sharing the pod IP and the app-logs volume.

Pattern	Example
Init container	Runs to completion before the app: wait for the DB, run migrations, fetch config, fix volume permissions
Sidecar	Runs beside the app for its whole life: Fluent Bit log shipper, Envoy or Istio proxy, Vault agent
Ambassador	Local proxy to an outside service, such as the Cloud SQL Auth Proxy
Adapter	Converts the app's output to a standard form, such as a Prometheus exporter

Since v1.33, native sidecars are stable: an init container with `restartPolicy: Always` starts before the app, keeps running beside it and stops after it.

```
apiVersion: v1
kind: Pod
metadata:
  name: orders-api
```

```

  labels: { app: orders-api }
spec:
  initContainers:
    - name: wait-for-db                                # classic init: runs once, must exit
      0
      image: busybox:1.36
      command: ['sh', '-c', 'until nc -z postgres 5432; do echo waiting; sleep
2; done']
    - name: log-shipper                                # native sidecar
      image: fluent/fluent-bit:3.0                    # its config comes from a ConfigMap,
omitted here
      restartPolicy: Always
      volumeMounts:
        - { name: app-logs, mountPath: /var/log/app }
  containers:
    - name: app
      image: myorg/orders-api:1.4.2
      ports: [{ containerPort: 8080 }]
      volumeMounts:
        - { name: app-logs, mountPath: /var/log/app }
  volumes:
    - name: app-logs
      emptyDir: {}

```

```

kubectl describe pod orders-api                        # Init Containers and
Containers sections
kubectl logs orders-api -c app
kubectl exec -it orders-api -c app -- sh

```

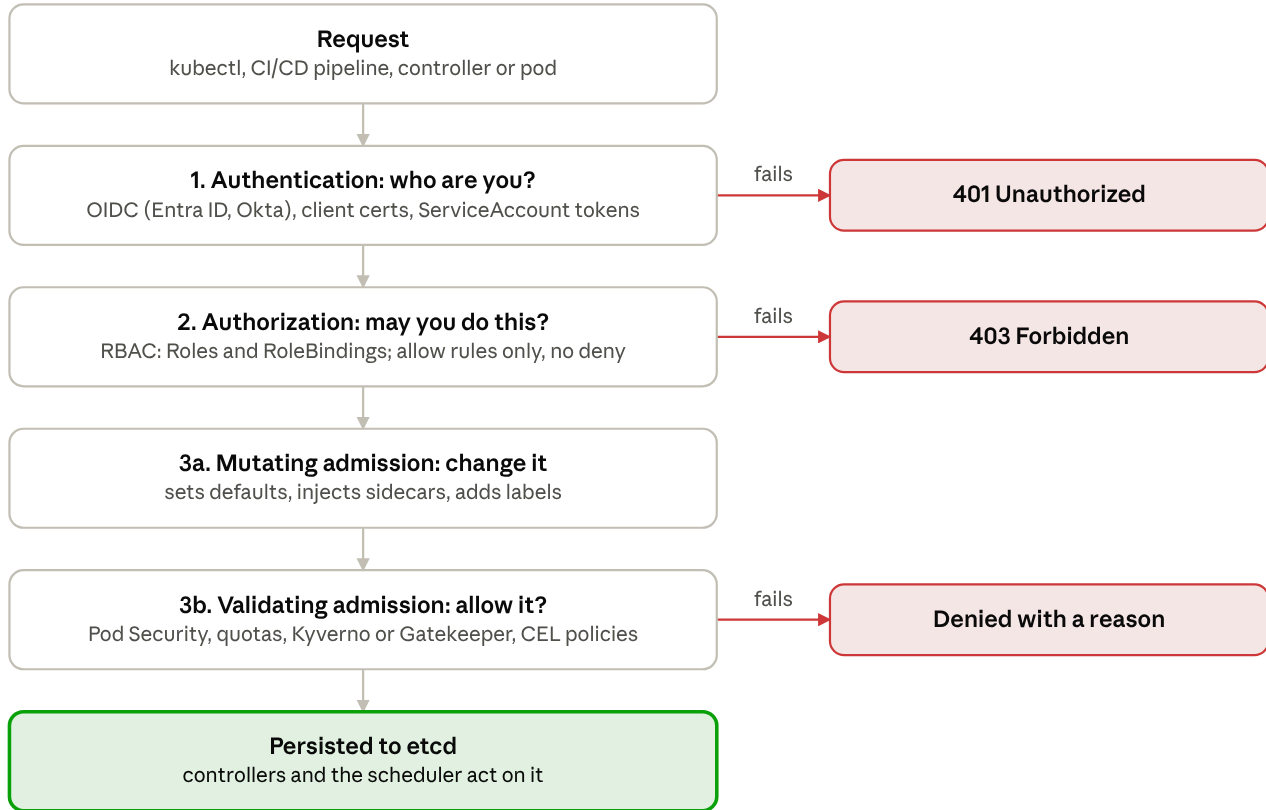
Rule of thumb: share a pod only when containers must share a lifecycle and a host. An app and its database belong in separate pods, because they scale and fail independently.

Q16. How does access control work in Kubernetes?

Every request to the API server passes three gates in order: authentication (who are you), authorization (may you do this, usually RBAC) and admission control (is this object allowed, and does it need changing). Fail any gate and the request is rejected before anything reaches etcd.

Every API request passes three gates before anything reaches etcd

Red exits: the request is refused and nothing is written



API request flow · authentication, authorization, admission

Failing authentication returns 401, failing RBAC returns 403, and a policy rejection returns the policy's message; only a request that clears every gate is stored.

1. Authentication. Kubernetes has no user objects; identities come from outside.

- Humans: OIDC from the corporate identity provider (Entra ID, Okta, Keycloak), EKS access entries mapped from AWS IAM, AKS with Entra ID, or client certificates for break-glass access only.
- Workloads: ServiceAccounts, whose tokens are short-lived and bound to the pod.

2. Authorization (RBAC).

Object	Scope	Purpose
Role	One namespace	A set of allowed verbs on resources

Object	Scope	Purpose
ClusterRole	Cluster-wide, or reused inside namespaces	The same, for cluster resources (nodes, PVs) or shared roles
RoleBinding	One namespace	Grants a Role or ClusterRole to users, groups or ServiceAccounts
ClusterRoleBinding	Whole cluster	Grants a ClusterRole everywhere; use sparingly

Verbs are `get`, `list`, `watch`, `create`, `update`, `patch` and `delete`, plus the dangerous `escalate`, `bind` and `impersonate`. RBAC is additive only: there are no deny rules.

Example 1: developers get read-only access to pods and logs in `dev`, through their identity-provider group.

```

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
rules:
- apiGroups: ['']
  resources: ['pods', 'pods/log', 'services', 'configmaps']
  verbs: ['get', 'list', 'watch']
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dev-team-read
  namespace: dev
subjects:
- kind: Group
  name: dev-team                                # group claim from OIDC / Entra ID
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io

```

Example 2: a CI/CD ServiceAccount that can deploy only to `prod`.

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: cicd-deployer
  namespace: prod
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deployer
  namespace: prod
rules:
- apiGroups: ['apps']
  resources: ['deployments']
  verbs: ['get', 'list', 'watch', 'create', 'update', 'patch']
- apiGroups: ['']
  resources: ['services', 'configmaps']
  verbs: ['get', 'list', 'create', 'update', 'patch']
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: cicd-deployer
  namespace: prod
subjects:
- kind: ServiceAccount
  name: cicd-deployer
  namespace: prod
roleRef:
  kind: Role
  name: deployer
  apiGroup: rbac.authorization.k8s.io

```

Verify before and after:

```

kubectl auth can-i delete pods -n dev --as=jane --as-group=dev-team      #
no
kubectl auth can-i patch deployments -n prod --
as=system:serviceaccount:prod:cicd-deployer    # yes
kubectl auth can-i --list -n prod

```

3. Admission control. Mutating admission changes the object (injects a sidecar, adds default labels); validating admission accepts or rejects it.

- Built in: `ResourceQuota`, `LimitRanger` and Pod Security Admission (namespace label `pod-security.kubernetes.io/enforce: restricted`).
- Policy engines: Kyverno or OPA Gatekeeper (block `:latest` tags, require resource limits, allow only the company registry), or ValidatingAdmissionPolicy with CEL rules, stable since v1.30.

Around the API: NetworkPolicies control pod-to-pod traffic, workload identity (EKS Pod Identity or IRSA, Azure Workload Identity) controls what pods may do in the cloud, and audit logs record who did what.

Best practices: least privilege; groups, not individual users; one ServiceAccount per app; `automountServiceAccountToken: false` where the API is not needed; no day-to-day `cluster-admin` for humans. Treat `list` on Secrets as read access to every secret in that namespace.

Q17. What is kube-proxy?

kube-proxy is a node agent, usually run as a DaemonSet, that makes Services work: it watches Services and EndpointSlices and programs each node's kernel rules so traffic sent to a Service IP is forwarded to a ready pod. Despite the name, in its normal modes it is not in the data path; the kernel forwards the packets.

kube-proxy · control path and packet path across two nodes

kube-proxy only watches the API server and rewrites the node's rules; the packet itself goes from Pod A through those kernel rules to Pod B on another node.

Packet path for `curl http://orders-api.prod:80` from a pod:

1. CoreDNS resolves `orders-api.prod.svc.cluster.local` to the ClusterIP, say `10.96.12.34`.
2. The packet hits the node's rules, which match the ClusterIP and port.
3. A per-Service rule set picks one ready endpoint (at random in iptables mode).
4. DNAT rewrites the destination to the pod, say `10.244.2.7:8080`, and the CNI routes it, to another node if needed.
5. Connection tracking reverses the NAT on the replies automatically.

Mode	How it works	Status
<code>iptables</code>	NAT rule chains, random backend choice	Long-time default; rule updates slow down with thousands of Services
<code>ipvs</code>	Kernel L4 load balancer with hash tables	Deprecated in v1.35; planned to be off by default in v1.40 and removed in v1.43 (KEP-5495)
<code>nftables</code>	nftables sets and maps	Stable since v1.33; the intended replacement for both iptables and IPVS
eBPF replacement	Cilium or Calico eBPF data plane	No kube-proxy at all; best performance and visibility

```
kubectl get ds kube-proxy -n kube-system
kubectl get cm kube-proxy -n kube-system -o yaml | grep 'mode:'
kubectl get endpointslices -n prod -l kubernetes.io/service-name=orders-api

# on a node
sudo iptables -t nat -L KUBE-SERVICES -n | grep orders-api      # iptables mode
sudo nft list table ip kube-proxy                               # nftables mode
sudo ipvsadm -Ln                                              # IPVS mode
```

Moving an IPVS cluster to nftables is a ConfigMap change (`mode: nftables`) plus `kubectl rollout restart -n kube-system daemonset/kube-proxy` ; test it in a lower environment first.

Service not reachable? Check in this order: the Service selector matches the pod labels → the EndpointSlice has ready addresses → `targetPort` matches the container port → kube-proxy pods are healthy → a NetworkPolicy is not blocking the traffic → DNS resolves the name.

Q18. How do you get and manage a static IP in Kubernetes?

First ask which IP: pod IPs are temporary by design, so stability comes from Service IPs, DNS names, and cloud IPs that you reserve yourself for inbound and outbound traffic. The rule: reserve the IP outside Kubernetes (Terraform), then attach it, so deleting a Service never releases it.

Need	Solution
A stable address inside the cluster	The Service ClusterIP or DNS name (<code>orders-api.prod.svc.cluster.local</code>); a StatefulSet with a headless Service gives each pod a stable DNS name (<code>mysql-0.mysql.db.svc.cluster.local</code>)
A fixed public IP for inbound traffic	Reserve a static IP in the cloud and attach it to the LoadBalancer Service of your Ingress or Gateway controller; point DNS at it
A fixed source IP for outbound traffic (partner allow-lists)	A NAT gateway with a reserved IP, or an egress gateway
A fixed pod IP	Avoid it. Some CNIs allow it (Calico IP annotations) for legacy cases; prefer an egress gateway

Inbound on Azure AKS: create a Standard static public IP, then name it on the Service.

```
az network public-ip create -g MC_rg-prod_aks-prod_centralindia -n ingress-pip --sku Standard --allocation-method Static
```

```
metadata:
  annotations:
    service.beta.kubernetes.io/azure-pip-name: ingress-pip
    # IP kept in another resource group? add:
    # service.beta.kubernetes.io/azure-load-balancer-resource-group: rg-network
spec:
  type: LoadBalancer
```

If the IP lives outside the node resource group, the cluster identity needs Network Contributor rights on that group.

Inbound on AWS EKS (Network Load Balancer through the AWS Load Balancer Controller), one Elastic IP per public subnet:

```

metadata:
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: external
    service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing
    service.beta.kubernetes.io/aws-load-balancer-subnets: subnet-0aaa,subnet-0bbb
    service.beta.kubernetes.io/aws-load-balancer-eip-allocations: eipalloc-0123,eipalloc-0456

```

An Application Load Balancer has no static IPs: put AWS Global Accelerator in front of it, or use an NLB. On GKE, reserve a global address and reference it from GKE Ingress with the annotation `kubernetes.io/ingress.global-static-ip-name: web-ip`.

On-premises with MetalLB:

```

apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: prod-pool
  namespace: metallb-system
spec:
  addresses:
  - 10.20.30.100-10.20.30.120
  ---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: prod-l2
  namespace: metallb-system
spec:
  ipAddressPools: [prod-pool]

```

Pin a Service to one address with the annotation `metallb.io/loadBalancerIPs: 10.20.30.100` (older releases use the `metallb.universe.tf/` prefix). The generic `spec.loadBalancerIP` field has been deprecated since v1.24 because each provider treated it differently; use the provider's annotation instead.

Outbound: by default pods leave through their node's IP, which changes as nodes scale. Route egress through a NAT gateway with a reserved IP (AWS NAT Gateway with an Elastic IP, Azure NAT Gateway, GCP Cloud NAT with reserved addresses), or use an egress gateway (Istio, Cilium Egress Gateway, Calico, OpenShift EgressIP) to give specific namespaces a fixed source IP.

Managing them: keep IPs in Terraform with `prevent_destroy`, tag them by environment, let ExternalDNS maintain the DNS records, document which partner allow-lists depend on each IP, and alert if a Service's external IP ever changes.

Q19. How do you manage secrets in Kubernetes?

Native Secrets are only base64-encoded, not encrypted, so production setups keep the source of truth in a secret manager, sync it into the cluster, and lock down etcd and RBAC. Git never holds a plain secret value.

```
kubectl create secret generic db-cred -n prod --from-literal=username=app --
from-literal=password='S3cr3t!'
kubectl get secret db-cred -n prod -o jsonpath='{.data.password}' | base64 -d
# anyone with 'get' can read it
```

Layer 1: harden native Secrets.

- Encrypt them at rest in etcd with a KMS provider (`EncryptionConfiguration`, KMS v2): AWS KMS on EKS, Azure Key Vault KMS on AKS, Cloud KMS on GKE.
- RBAC: no `get`, `list` or `watch` on Secrets for humans in production; `list` returns every value.
- Prefer mounted files to environment variables: files update when the Secret changes and do not leak into child processes or crash dumps.
- `immutable: true` for secrets that never change, separate namespaces per app and environment, and audit logging on Secret reads.

Layer 2: an external secret manager (the production answer).

Tool	How it works	Pick it when
External Secrets Operator (ESO)	Syncs values from AWS Secrets Manager, Azure Key Vault, HashiCorp Vault or GCP into native Secrets on a refresh interval	Apps already read normal Secrets, and vault rotations should flow through
Secrets Store CSI Driver	Mounts secrets from the vault as files at pod start, with optional sync to a Secret	Secrets should never be stored in etcd

Tool	How it works	Pick it when
HashiCorp Vault Agent Injector or Vault Secrets Operator	Injects files through a sidecar, or syncs; supports dynamic database credentials with TTLs	Vault is the company standard, or you need short-lived credentials
Sealed Secrets or SOPS	Encrypts the secret so the encrypted file can live in Git; decrypted in the cluster (controller) or at deploy time (SOPS with KMS or age)	GitOps with Argo CD or Flux and no external vault

Secrets flow · Git, External Secrets Operator, secret manager, Secret, pod

Git carries only the ExternalSecret reference; the operator reads the value from the secret manager with workload identity, writes a KMS-encrypted Secret, and the pod consumes it.

The operator and the pods authenticate to the vault with workload identity (EKS Pod Identity or IRSA, Azure Workload Identity, GKE Workload Identity, or Vault's Kubernetes auth), so no static cloud keys sit in the cluster.

ESO example with AWS Secrets Manager:

```

apiVersion: external-secrets.io/v1          # v1beta1 on older ESO releases
kind: ClusterSecretStore
metadata:
  name: aws-sm
spec:
  provider:
    aws:
      service: SecretsManager
      region: ap-south-1
      auth:
        jwt:
          serviceAccountRef:                # ServiceAccount mapped to an IAM
            role (IRSA)
              name: external-secrets
              namespace: external-secrets
---
```

```

apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: orders-db
  namespace: prod
spec:
  refreshInterval: 1h                                # picks up rotations
  secretStoreRef:
    name: aws-sm
    kind: ClusterSecretStore
  target:
    name: orders-db-secret                            # native Secret that ESO creates
and owns
data:
  - secretKey: DB_PASSWORD
    remoteRef:
      key: prod/orders/db
      property: password

```

The Deployment consumes it like any Secret:

```

env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: orders-db-secret
      key: DB_PASSWORD

```

Rotation: environment variables are read only at start-up, so pair rotation with a rollout (Stakater Reloader restarts the Deployment when the Secret changes), or read mounted files, which update in place.

In the interview: "Secrets live in AWS Secrets Manager, one path per environment. External Secrets Operator syncs them using IRSA, etcd is encrypted with KMS, developers have no Secret read access in prod, and Reloader rolls pods on rotation. Git holds only ExternalSecret manifests."

Part 4 · Ansible

Q20. What are Ansible roles?

A role is Ansible's unit of reuse: a fixed folder layout that bundles the tasks, handlers, variables, templates and files for one job, such as installing Nginx, hardening an OS or deploying an app. Ansible loads each folder by convention, so a playbook only has to name the role.

```
roles/nginx/                                # created with: ansible-galaxy role init
roles/nginx
├─ defaults/main.yml                        # default variables, lowest precedence: meant to
be overridden
├─ vars/main.yml                            # role constants, high precedence: not meant to
be overridden
├─ tasks/main.yml                          # entry point for the tasks
├─ handlers/main.yml                      # run once, after the tasks, only when notified
├─ templates/site.conf.j2                 # Jinja2 templates
├─ files/                                 # static files to copy
├─ meta/main.yml                          # metadata, supported platforms, role
dependencies
├─ tests/                                 # sample inventory and playbook
└─ README.md
```

```
# roles/nginx/tasks/main.yml
- name: Install nginx
  ansible.builtin.package:
    name: nginx
    state: present

- name: Deploy site config
  ansible.builtin.template:
    src: site.conf.j2
    dest: /etc/nginx/conf.d/site.conf
    mode: '0644'
  notify: Reload nginx

- name: Start and enable nginx
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: true
```

```
# roles/nginx/handlers/main.yml
- name: Reload nginx
  ansible.builtin.service:
    name: nginx
    state: reloaded

# roles/nginx/defaults/main.yml
nginx_listen_port: 80
nginx_server_name: shop.example.com
```

```
{# roles/nginx/templates/site.conf.j2 #}
server {
    listen {{ nginx_listen_port }};
    server_name {{ nginx_server_name }};
    location / { proxy_pass http://127.0.0.1:8080; }
}
```

Using roles:

```
# site.yml
- hosts: webservers
  become: true
  roles:
    - common
    - role: nginx
    vars:
      nginx_listen_port: 8080

# or from inside tasks
tasks:
  - name: Static import, resolved when the playbook is parsed (tags and --
list-tasks see inside)
    ansible.builtin.import_role:
      name: nginx
  - name: Dynamic include, resolved at run time (works with loops and
conditions)
    ansible.builtin.include_role:
      name: app_deploy
      when: deploy_app | bool
```

Dependencies and sharing:

```
# roles/nginx/meta/main.yml
dependencies:
  - role: common

# requirements.yml, installed with: ansible-galaxy install -r
requirements.yml
roles:
  - name: geerlingguy.docker                # from Ansible Galaxy
  - src: https://git.example.com/infra/ansible-role-hardening.git
    scm: git
    version: v1.2.0                        # pin to a tag
    name: hardening
collections:
  - name: community.general
```

Roles vs collections: a collection (`namespace.name`) packages roles together with modules and plugins, and is now the standard way to distribute Ansible content.

Quality: test roles with Molecule (creates containers or VMs, converges the role, verifies, then checks idempotence by running it twice) and lint with `ansible-lint` in CI.

In the interview: "We kept one role per concern (common, hardening, java, app_deploy), defaults for everything, environment values in inventory group_vars, Molecule tests in CI, and versions pinned in requirements.yml."

Q21. What is Ansible Vault?

Ansible Vault encrypts sensitive data with AES-256, either whole files or single variables, so it can be committed to Git; Ansible decrypts it in memory at run time when given the vault password.

```
ansible-vault create  group_vars/prod/vault.yml      # new encrypted file,
opens $EDITOR
ansible-vault edit    group_vars/prod/vault.yml
ansible-vault view    group_vars/prod/vault.yml
ansible-vault encrypt secrets.yml                    # encrypt an existing
file
ansible-vault decrypt secrets.yml                    # back to plain text
(never commit that)
ansible-vault rekey   group_vars/prod/vault.yml      # change the password
ansible-vault encrypt_string 'S3cr3t!' --name 'vault_db_password' # one
encrypted variable
```

An encrypted file is a header plus hex ciphertext; `encrypt_string` output is pasted straight into a vars file:

```
# whole file
$ANSIBLE_VAULT;1.1;AES256
62313365396662343061393464336163383764373764613633653634306231386433626436623
361

# single variable
vault_db_password: !vault |
    $ANSIBLE_VAULT;1.1;AES256
    3961336336323138356537663934...
```

Running with Vault:

```
ansible-playbook site.yml --ask-vault-pass
ansible-playbook site.yml --vault-password-file ~/.vault_pass # a
file, or an executable script
ansible-playbook site.yml --vault-id dev@~/.vault_dev --vault-id prod@prompt
```

Vault IDs give each environment its own password. Encrypt with `ansible-vault encrypt --vault-id prod@prompt group_vars/prod/vault.yml` and the header records the label (`$ANSIBLE_VAULT;1.2;AES256;prod`), so a dev engineer cannot decrypt production secrets.

The vars/vault pattern keeps variable names searchable while the values stay encrypted:

```
# group_vars/prod/vars.yml (plain text)
db_user: app
db_password: '{{ vault_db_password }}'

# group_vars/prod/vault.yml (encrypted)
vault_db_password: S3cr3t!
```

Good practice:

- Put `no_log: true` on tasks that handle secrets, so values never reach the console or AWX job logs.
- In CI (Jenkins, GitHub Actions), store the vault password as a pipeline credential and pass it with `--vault-password-file` or `ANSIBLE_VAULT_PASSWORD_FILE`; add `.vault_pass` to `.gitignore`.

- The password file can be a script that fetches the password from AWS Secrets Manager, HashiCorp Vault or CyberArk.
- For secrets that rotate often, skip Vault and look them up at run time: `community.hashi_vault.vault_kv2_get`, the `amazon.aws.secretsmanager_secret` lookup, or the Azure Key Vault lookup.
- Ansible Vault is file encryption, not a secrets manager: it has no rotation, audit trail or access policy. Rekey when people with the password leave.

Q22. How do you manage variables for different environments in Ansible?

Give each environment its own inventory directory with its own `group_vars` and `host_vars`, keep safe defaults in roles, and run the same playbook with a different `-i`. Playbooks and roles never change between dev and prod; only the inventory and its variables do.

```

ansible/
├─ ansible.cfg
├─ site.yml
├─ requirements.yml
├─ roles/
│   └─ common/
│       └─ app_deploy/                # defaults/main.yml holds safe defaults
├─ inventories/
│   └─ dev/
│       ├── hosts.yml
│       └─ group_vars/
│           ├── all.yml              # env: dev, log_level: debug
│           └─ webservers.yml
├─ qa/
│   └─ ...
├─ prod/
│   ├── hosts.yml
│   ├── group_vars/
│   │   ├── all/
│   │   │   └─ vars.yml             # env: prod, log_level: warn, db_password:
'{{ vault_db_password }}'
│   │   └─ vault.yml               # encrypted with --vault-id prod
│   │   └─ webservers.yml
├─ host_vars/
│   └─ web01.yml                   # host-specific overrides

```

```
# inventories/prod/hosts.yml
all:
  children:
    webservers:
      hosts:
        web01: { ansible_host: 10.10.1.11 }
        web02: { ansible_host: 10.10.1.12 }
    dbservers:
      hosts:
        db01: { ansible_host: 10.10.2.21 }
```

```
ansible-playbook -i inventories/dev site.yml
ansible-playbook -i inventories/prod site.yml --vault-id prod@prompt
ansible-inventory -i inventories/prod --graph           # groups and hosts
ansible-inventory -i inventories/prod --host web01      # final merged
variables for one host
```

Other options, and when they fit:

- `-e app_version=1.4.2` or `-e @release.yml` for values the pipeline decides per run (image tag, build number); extra vars always win.
- `vars_files: ['vars/{{ env }}.yaml']` with `-e env=prod`, in small projects with a single inventory.
- Dynamic inventory (`amazon.aws.aws_ec2`, `azure.azcollection.azure_rm`) with `keyed_groups` built from an `Environment` tag, so `group_vars/env_prod.yml` applies automatically.
- In AWX or Ansible Automation Platform: one inventory per environment, with credentials attached to the job template.

Variable precedence, lowest to highest (simplified from the official 22 levels):

1. Role `defaults/main.yml`
2. Inventory `group_vars/all`, then `group_vars/<group>`; child groups override parents, and `group_vars` beside the playbook override those beside the inventory
3. Inventory `host_vars/<host>`
4. Facts gathered from the host
5. Play `vars` and `vars_files`
6. Role `vars/main.yml`
7. Block and task `vars`, then `include_vars`

8. `set_fact` and registered variables
9. Role and include parameters
10. Extra vars (`-e`), which always win

Two traps to mention: a value an environment may change belongs in role `defaults`, because role `vars` would beat your inventory; and when a value surprises you, check the merged result with `ansible-inventory --host` or `-m ansible.builtin.debug -a var=<name>`.

Q23. How do you run an Ansible playbook, and what are the options?

The basic run is `ansible-playbook -i <inventory> <playbook>.yaml`; the options control which hosts, which tasks, how safely and with which credentials. In production, always rehearse with `--check --diff` on a limited set of hosts first.

```
ansible-playbook -i inventories/prod site.yaml
```

Option	What it does	Example
<code>-i, --inventory</code>	Inventory file, directory, script or plugin	<code>-i inventories/prod</code>
<code>-l, --limit</code>	Run on a subset of hosts or groups	<code>-l web01, -l 'webservers:!web03'</code>
<code>-e, --extra-vars</code>	Set variables (highest precedence)	<code>-e app_version=1.4.2, -e @release.yaml</code>
<code>-C, --check</code>	Dry run: report what would change	<code>--check</code>
<code>-D, --diff</code>	Show file and template differences	<code>--check --diff</code>
<code>-t, --tags / --skip-tags</code>	Run only, or skip, tagged tasks	<code>-t nginx, --skip-tags db</code>
<code>--start-at-task</code>	Resume from a named task	<code>--start-at-task 'Deploy site config'</code>
<code>--step</code>	Confirm each task interactively	<code>--step</code>

Option	What it does	Example
<code>--syntax-check</code>	Parse only, run nothing	<code>--syntax-check</code>
<code>--list-hosts</code> / <code>--list-tasks</code> / <code>--list-tags</code>	Show what would run	<code>--list-hosts -l webservers</code>
<code>-b</code> , <code>--become</code>	Escalate privileges (sudo)	<code>-b --become-user root</code>
<code>-K</code> , <code>--ask-become-pass</code>	Prompt for the sudo password	<code>-K</code>
<code>-u</code> , <code>--user</code> / <code>--private-key</code>	SSH user and key	<code>-u deploy --private-key ~/.ssh/prod.pem</code>
<code>-k</code> , <code>--ask-pass</code>	Prompt for an SSH password	<code>-k</code>
<code>-f</code> , <code>--forks</code>	Hosts handled in parallel (default 5)	<code>-f 20</code>
<code>--vault-id</code> / <code>--ask-vault-pass</code> / <code>--vault-password-file</code>	Decrypt Vault data	<code>--vault-id prod@prompt</code>
<code>-c</code> , <code>--connection</code>	Connection type	<code>-c local</code> , <code>-c winrm</code>
<code>-v</code> to <code>-vvvv</code>	More output; <code>-vvvv</code> adds connection debugging	<code>-vvv</code>

A safe production sequence:

```

ansible-lint site.yml
ansible-playbook -i inventories/prod site.yml --syntax-check
ansible-playbook -i inventories/prod site.yml -l webservers --list-hosts
ansible-playbook -i inventories/prod site.yml -l webservers -t nginx --check
--diff --vault-id prod@prompt
ansible-playbook -i inventories/prod site.yml -l webservers -t nginx --vault-id prod@prompt

```

Rolling deploys are set inside the playbook, not on the command line:

```

- name: Rolling deploy of the orders app
  hosts: webservers
  become: true
  serial: '25%'                # a quarter of the hosts per batch
  max_fail_percentage: 10      # stop the rollout if more than 10% of a
batch fails
  roles:
    - { role: common, tags: [common] }
    - { role: app_deploy, tags: [app, deploy] }

```

Related tools worth naming:

- Ad-hoc commands for one-off actions: `ansible webservers -i inventories/prod -m ansible.builtin.ping`, or `-m ansible.builtin.shell -a 'df -h' -b`.
- `ansible.cfg` for defaults such as the inventory path, forks and `pipelining = True` (fewer SSH round trips, faster runs).
- `ansible-navigator run site.yml` with execution environments (container images holding Ansible and its collections) in Ansible Automation Platform.
- AWX or Automation Platform job templates for RBAC, schedules, surveys and an audit trail of every run.